

SAPIENZA
UNIVERSITÀ DI ROMA

AI Lab: Computer Vision and NLP

Bachelor Degree in Applied Computer Science and Artificial Intelligence

Posture Correction of Exercises Using MediaPipe and Random Forest Classifiers

Thiago Cleto Miarelli Piedade
Aleksandra Byczkowska
Betul Gul
Ayca Su Alparslan

June 2024

1 Introduction

Our project aims to develop an exercise recognition and correction system that will provide personalized feedback to people at all fitness levels, since physical exercise is an essential aspect of a healthy lifestyle, but the guidance of a personal trainer can be both costly and not accessible to everyone. The objective is to enable a webcam to detect and analyze human body movements during various exercises, recognize specific exercises, and provide real-time feedback to correct the posture if necessary. The system is capable of tracking key body landmarks, calculating joint angles, and comparing these angles with predefined ranges for different exercises to ensure effective workouts.

Our approach combines the feed from a standard webcam with a machine learning model that can identify and analyze exercises in real-time.

2 Tracking body points

Tracking the most important body joints, such as elbows, shoulders, hips and knees is the first step in order for the program to recognize the exercise that is being performed. It consists of identifying the key body joints and computing the angle between them.

Each frame from the webcam or a video file is processed and analyzed using MediaPipe, a machine learning framework for pose estimation, that enables to identify the key body landmarks mentioned before. Each landmark has x and y coordinates representing its position in the 2D image plane. We also defined the important angles that have to be monitored during every exercise. The calculation for each angle is done by getting one axis composed of two points, such as the vector made by the ray between the landmarks of the right hip and the right knee, and a normal line, which is always orthogonal to the top border. We call this normal line “reference axis”. s:

- Vector 1 (Knee to Hip): v_1=Knee Position−Hip Position $v_1=(x_2-x_1,y_2-y_1)$
- Vector 2 (Hip to Reference): v2=Hip Position−Reference Axis $v_2=(x_2-x_2,y_2-(y_2 - 20))$

The cosine of the angle between 2 vectors is given by:

$$cos\,(\Theta) = \frac{v1 * v2}{||v1|| * ||v2||}$$

From this we can obtain the angle:

$$\Theta = arccos\,(\frac{v1 * v2}{||v1|| * ||v2||})$$

And convert it from radians to degrees, to make the interpretation easier.

3 Identifying the Exercise

The angle calculation described above to correct exercises is also used in the prediction algorithm that classifies which movement the user is doing. Identifying the exercise was fundamental to the pose correction aimed as the project result. Clearly, different exercises required different correction values.

The training process started with the download of a large dataset of labeled exercise photos. Then, we runned the tracking algorithm to detect the body points in the image and computed the angles of the main articulations.

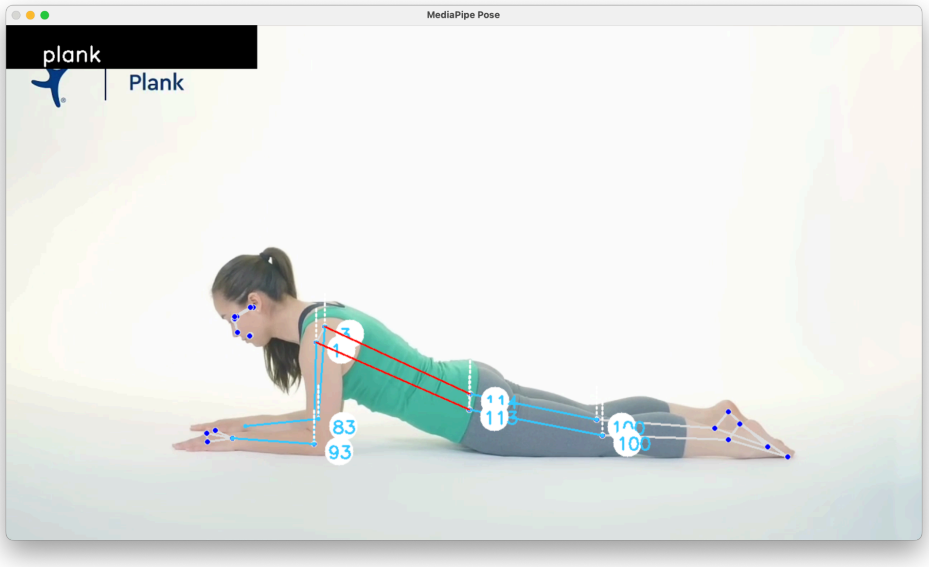


Image 1: Tracked angles in a wrong plank

	right_knee	left_knee	right_hip	left_hip	right_shoulder	left_shoulder	right_elbow	left_elbow	label
2	169.238206	177.671552	164.561810	177.053121	179.926015	165.142456	76.821728	111.725114	5
3	62.627412	106.539886	90.009577	110.081440	22.710753	151.001158	28.633834	36.150020	5
8	34.995963	167.917890	157.184597	151.335476	133.045401	112.491120	155.291014	152.523240	5
42	174.742875	178.469169	178.667531	173.131412	115.792893	86.489348	162.252904	176.942402	5
45	155.652732	161.280042	160.295361	175.040569	110.975737	91.107006	146.831028	36.899530	5

Image 2: Processed samples with their angles and labels

The angles along with the label were written in a dataframe. With the dataframe in hands, we performed a dimensionality reduction and plotting to understand how different the data between each exercise was.

Index	0	1	2	3
Exercise	plank	romanian deadlift	russian twist	squat

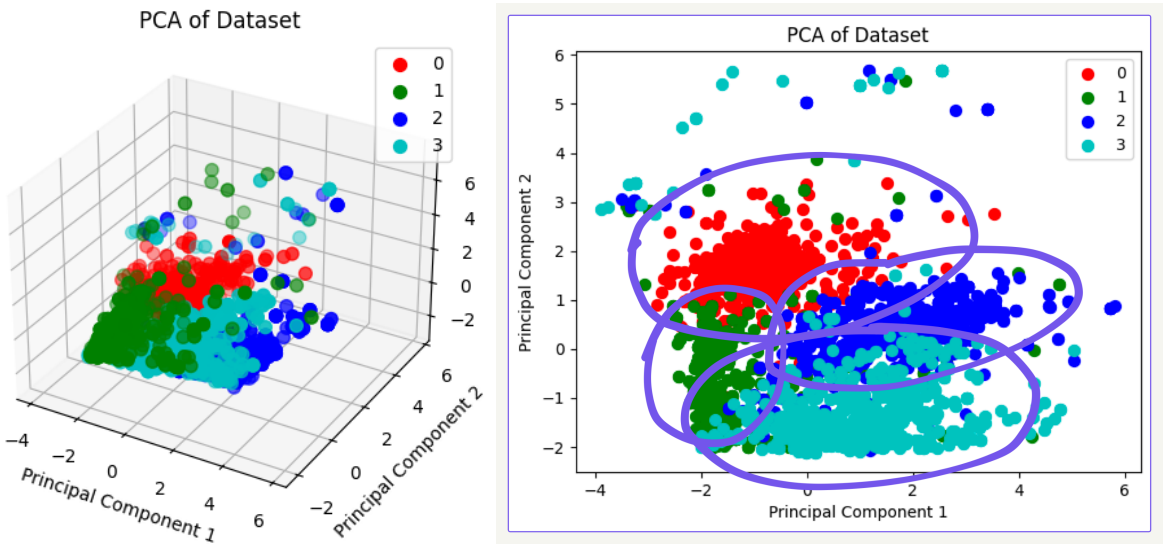


Image 3: PCA dimensionality reduction in the post-processed dataset of exercise angles

It is clear that there is meaningful information encoded in the angles, therefore, we proceeded into using that information in our classifier. After testing in many models like SVMs and MLPs, the best result was reached by a Random Tree Classifier. We've achieved a 0.96 precision in the test set, and a great confusion matrix:

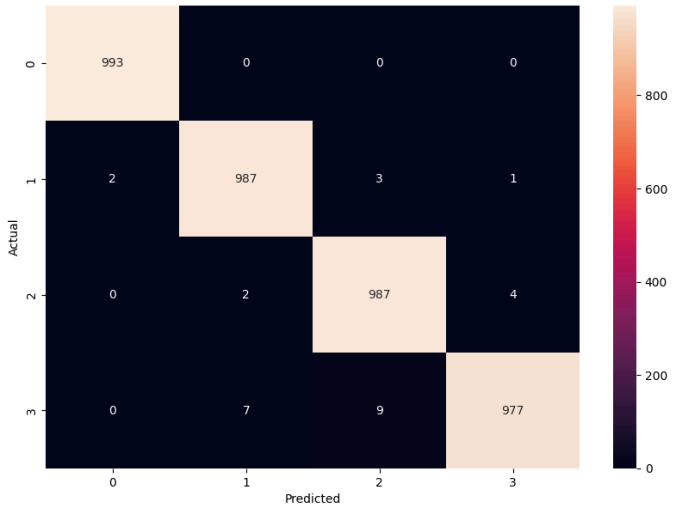


Image 4: Random Forest Classifier Confusion Matrix

3.1 Implementation Details

Due to the data intensive nature of video processing contrasting with the relatively slow pace of exercises, two main challenges were presented in the implementation phase: efficiency and prediction stability.

The exercise recognition and correction system ensures efficiency by not making predictions for every single frame. Instead, predictions are generated at specific intervals defined by the `freq_predict` parameter. This means that the system only makes a prediction every few frames, which significantly reduces the computational load. By spacing out predictions, the system can process frames more efficiently, ensuring that real-time feedback is provided without overwhelming the processing resources.

To maintain stability, the system averages predictions over several frames and evaluates them at broader intervals defined by `freq_update * freq_predict` frames. This cumulative approach stores prediction probabilities in an array called `cumulative_predictions`. Every evaluation interval, the system checks if the maximum probability exceeds a threshold of 0.5. If this threshold is met, the current exercise is updated to the one with the highest probability. This method ensures that the exercise is only updated when the system is confident, avoiding erratic changes and providing consistent and reliable feedback.

4 Real-time feedback mechanism and user interface

Once the exercise is identified, the system checks each joint angle against predefined acceptable ranges for the current exercise, that we obtained before based on the dataset. If any joint angle falls outside these ranges, the joint is labeled as incorrect and highlighted in red to draw the user's attention to the areas that need correction.

The detected pose landmarks and calculated angles, together with the name of the identified exercise, are visualized directly on the video feed using OpenCV, which makes it very easy for the user to understand their body posture and the angles that are being monitored and provides an immediate guidance

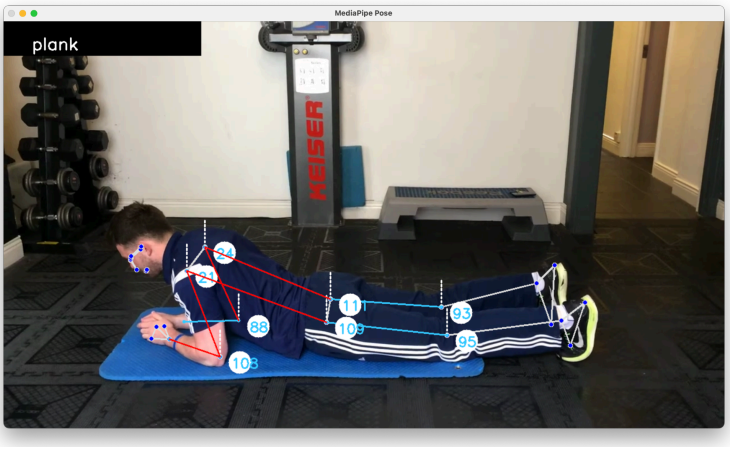


Image 5: Wrong posture of a plank with many red lines

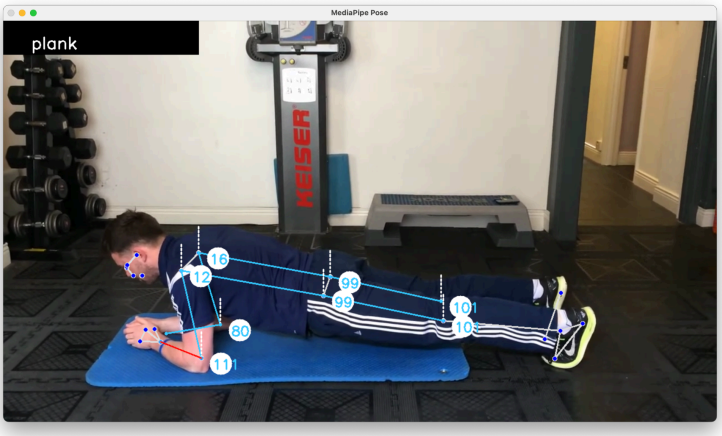


Image 6: Correct posture of a plank

5 Conclusions

The development of a real-time exercise recognition and correction system required us to try different approaches and various machine learning and computer vision algorithms, but after several attempts we can conclude that the use of a Random Forest Classifier together with the applied by us confidence threshold ensures stable and accurate exercise identification. We managed to create an accurate and efficient program with real-time feedback that makes the workout easier and more effective, which was our initial goal.

Bibliography

1.Vikram Sharma, Jianing Wei, Tyler Mullen, *Instant Motion Tracking with MediaPipe* (Aug 31, 2020). Retrieved from: https://developers.googleblog.com/en/instant-motion-tracking-with-mediapipe/

2. Kunal Dawn, *AI Fitness Trainer – Build Using MediaPipe For Squat Analysis* (Dec 20, 2022). Retrieved from: https://learnopencv.com/ai-fitness-trainer-using-mediapipe/

3. Google AI Developers, *Pose Landmark Detection Guide*. Retrieved from: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker?hl=pt-br

4. Hasym Abdillah, *Workout Exercises Images Dataset on Kaggl*e. Retrieved from: https://www.kaggle.com/datasets/hasyimabdillah/workoutexercises-images

5. scikit-learn, *RandomForestClassifier Documentation*. Retrieved from: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

6. Will Koehrsen, *Hyperparameter Tuning the Random Forest in Python using Scikit-Learn* (Sep 30, 2018). Retrieved from: https://towardsdatascience.com/hyperparameter-tuning-the-random-forest-in-python-using-scikit-learn-28d2aa77dd74